

# raw\_to\_silver

September 25, 2025

## 1 ETL da camada bronze para camada silver

Este notebook realiza o ETL dos dados da camada bronze para a camada silver. Ou seja: ele abre o dataset e o salva num dataframe, realiza a transformação dos dados e os carrega num arquivo .csv e no banco de dados.

### 1.1 EXTRACT

```
[1]: import pandas as pd
import numpy as np

data_layer_filepath = '../data_layer/'

df = pd.read_csv(data_layer_filepath + 'raw/airbnb-dataset.csv',
                 low_memory=False)
print("Dataset carregado com sucesso!")
df.head()
```

Dataset carregado com sucesso!

```
[1]:
```

|   | id      | NAME                                             | host id     | \           |
|---|---------|--------------------------------------------------|-------------|-------------|
| 0 | 1001254 | Clean & quiet apt home by the park               | 80014485718 |             |
| 1 | 1002102 | Skylit Midtown Castle                            | 52335172823 |             |
| 2 | 1002403 | THE VILLAGE OF HARLEM...NEW YORK !               | 78829239556 |             |
| 3 | 1002755 |                                                  | NaN         | 85098326012 |
| 4 | 1003689 | Entire Apt: Spacious Studio/Loft by central park | 92037596077 |             |

  

|   | host_identity_verified | host name | neighbourhood | group | neighbourhood | \ |
|---|------------------------|-----------|---------------|-------|---------------|---|
| 0 | unconfirmed            | Madaline  | Brooklyn      |       | Kensington    |   |
| 1 | verified               | Jenna     | Manhattan     |       | Midtown       |   |
| 2 | NaN                    | Elise     | Manhattan     |       | Harlem        |   |
| 3 | unconfirmed            | Garry     | Brooklyn      |       | Clinton Hill  |   |
| 4 | verified               | Lyndon    | Manhattan     |       | East Harlem   |   |

  

|   | lat      | long      | country       | ... | service fee | minimum nights | \ |
|---|----------|-----------|---------------|-----|-------------|----------------|---|
| 0 | 40.64749 | -73.97237 | United States | ... | \$193       | 10.0           |   |
| 1 | 40.75362 | -73.98377 | United States | ... | \$28        | 30.0           |   |
| 2 | 40.80902 | -73.94190 | United States | ... | \$124       | 3.0            |   |

|   |          |           |               |     |      |      |
|---|----------|-----------|---------------|-----|------|------|
| 3 | 40.68514 | -73.95976 | United States | ... | \$74 | 30.0 |
| 4 | 40.79851 | -73.94399 | United States | ... | \$41 | 10.0 |

  

|   | number of reviews | last review | reviews per month | review rate | number \ |
|---|-------------------|-------------|-------------------|-------------|----------|
| 0 | 9.0               | 10/19/2021  | 0.21              |             | 4.0      |
| 1 | 45.0              | 5/21/2022   | 0.38              |             | 4.0      |
| 2 | 0.0               | NaN         | NaN               |             | 5.0      |
| 3 | 270.0             | 7/5/2019    | 4.64              |             | 4.0      |
| 4 | 9.0               | 11/19/2018  | 0.10              |             | 3.0      |

  

|   | calculated host listings count | availability 365 \ |
|---|--------------------------------|--------------------|
| 0 | 6.0                            | 286.0              |
| 1 | 2.0                            | 228.0              |
| 2 | 1.0                            | 352.0              |
| 3 | 1.0                            | 322.0              |
| 4 | 1.0                            | 289.0              |

  

|   | house_rules                                       | license |
|---|---------------------------------------------------|---------|
| 0 | Clean up and treat the home the way you'd like... | NaN     |
| 1 | Pet friendly but please confirm with me if the... | NaN     |
| 2 | I encourage you to use my kitchen, cooking and... | NaN     |
| 3 |                                                   | NaN     |
| 4 | Please no smoking in the house, porch or on th... | NaN     |

[5 rows x 26 columns]

## 1.2 TRANSFORM

### 1.2.1 Padronização dos Nomes das Colunas.

Inicialmente, o dataset contém colunas com nomes sem padrão. Algumas colunas separam palavras com `_`, outras com `whitespace`. A coluna `NAME` também foge do padrão, visto que está com todos os caracteres em letra maiúscula. Por isso, o padrão novo de nomes de colunas será: **todos os caracteres em minúsculo, separando palavras com `_`**.

```
[2]: df.rename(
      columns={col: col.lower().replace(' ', '_') for col in df.columns},
      inplace=True
    )
print(df.columns)
```

```
Index(['id', 'name', 'host_id', 'host_identity_verified', 'host_name',
      'neighbourhood_group', 'neighbourhood', 'lat', 'long', 'country',
      'country_code', 'instant_bookable', 'cancellation_policy', 'room_type',
      'construction_year', 'price', 'service_fee', 'minimum_nights',
      'number_of_reviews', 'last_review', 'reviews_per_month',
      'review_rate_number', 'calculated_host_listings_count',
      'availability_365', 'house_rules', 'license'],
      dtype='object')
```

### 1.2.2 Remoção de Colunas Desnecessárias

Como quase todas as tuplas de **license** estavam com valor nulo, optamos por não trabalhar com essa coluna. Além disso, escolhemos remover as colunas **country** e **country\_code** visto que sabemos que todos os anúncios se enquadram no Estados Unidos (mais especificamente, na cidade de Nova York) e possuem o código do país como “US”.

```
[3]: cols_to_drop = ['country', 'country_code', 'license']
df.drop(columns=cols_to_drop, inplace=True)

for col in cols_to_drop:
    if col not in df.columns:
        print(f"Coluna {col} deletada!")

print("Colunas: ")
print(df.columns)
```

Coluna country deletada!

Coluna country\_code deletada!

Coluna license deletada!

Colunas:

```
Index(['id', 'name', 'host_id', 'host_identity_verified', 'host_name',
      'neighbourhood_group', 'neighbourhood', 'lat', 'long',
      'instant_bookable', 'cancellation_policy', 'room_type',
      'construction_year', 'price', 'service_fee', 'minimum_nights',
      'number_of_reviews', 'last_review', 'reviews_per_month',
      'review_rate_number', 'calculated_host_listings_count',
      'availability_365', 'house_rules'],
      dtype='object')
```

### 1.2.3 Correções dos Tipos de Dados.

- 1) **price** e **service\_fee** indicam valores, mas possuem o caracter especial “\$” e estão como object. Por isso, iremos alterá-las para o tipo numérico (float).

```
[4]: money_columns = ['price', 'service_fee']

print("Tipos de dados antes da correção:")
print(df[money_columns].dtypes)

for col in money_columns:
    df[col] = df[col].str.replace('$', '', regex=False).str.replace(',', '',
    ↪ regex=False).str.strip().astype(float)

print("Tipos de dados corrigidos:")
print(df[money_columns].dtypes)
```

Tipos de dados antes da correção:

price                      object

```

service_fee    object
dtype: object
Tipos de dados corrigidos:
price          float64
service_fee     float64
dtype: object

```

- 2) Colunas **construction\_year**, **minimum\_nights**, **number\_of\_reviews**, **availability\_365** e **calculated\_host\_listings\_count**: Essas colunas estão com o tipo float, mas elas indicam valores inteiros. Por isso, faremos a conversão do tipo delas. Além disso, removeremos os NaNs delas.

```

[5]: floats_to_ints = ['availability_365',
    ↪ 'construction_year', 'calculated_host_listings_count', 'number_of_reviews',
    ↪ 'minimum_nights']

for col in floats_to_ints:
    print(f"Tipo de dado da chave {col} antes da correção: {df[col].dtype}")

    df.dropna(subset=[col], inplace=True)
    df[col] = df[col].astype('int64')

    print(f"Tipo de dado da chave {col} após a correção: {df[col].dtype}")

```

```

Tipo de dado da chave availability_365 antes da correção: float64
Tipo de dado da chave availability_365 após a correção: int64
Tipo de dado da chave construction_year antes da correção: float64
Tipo de dado da chave construction_year após a correção: int64
Tipo de dado da chave calculated_host_listings_count antes da correção: float64
Tipo de dado da chave calculated_host_listings_count após a correção: int64
Tipo de dado da chave number_of_reviews antes da correção: float64
Tipo de dado da chave number_of_reviews após a correção: int64
Tipo de dado da chave minimum_nights antes da correção: float64
Tipo de dado da chave minimum_nights após a correção: int64

```

- 3) **host\_identity\_verified**: A coluna **host\_identity\_verified** tem apenas dois valores (como observado na análise de dados da camada bronze): **verified** e **unconfirmed**. Por isso, optamos por transformar esse tipo em **bool**, visto que sua informação é binária.

```

[6]: print("Tipo de dado da coluna ANTES do tratamento:")
    print(df['host_identity_verified'].dtype)
    print("\nValores únicos na coluna ANTES do tratamento:")
    print(df['host_identity_verified'].unique())
    print("\nContagem de valores na coluna ANTES do tratamento:")
    print(df['host_identity_verified'].value_counts(dropna=False))

df['host_identity_verified'] = df['host_identity_verified'].astype(str).str.
    ↪ lower().str.strip()

```

```

to_replace = {
    'verified': True,
    'unconfirmed': False
}

df['host_identity_verified'] = df['host_identity_verified'].replace(to_replace).
    ↪astype(bool)

print("Tipo de dado da coluna APÓS o tratamento:")
print(df['host_identity_verified'].dtype)
print("\nValores únicos na coluna APÓS o tratamento:")
print(df['host_identity_verified'].unique())
print("\nContagem de valores na coluna APÓS o tratamento:")
print(df['host_identity_verified'].value_counts(dropna=False))

```

Tipo de dado da coluna ANTES do tratamento:  
object

Valores únicos na coluna ANTES do tratamento:  
['unconfirmed' 'verified' nan]

Contagem de valores na coluna ANTES do tratamento:

| host_identity_verified |       |
|------------------------|-------|
| unconfirmed            | 50476 |
| verified               | 50403 |
| NaN                    | 255   |

Name: count, dtype: int64  
Tipo de dado da coluna APÓS o tratamento:  
bool

Valores únicos na coluna APÓS o tratamento:  
[False True]

Contagem de valores na coluna APÓS o tratamento:

| host_identity_verified |       |
|------------------------|-------|
| True                   | 50658 |
| False                  | 50476 |

Name: count, dtype: int64

4. **Colunas object:** algumas colunas tem o tipo misto object. Na análise realizada na camada bronze, concluímos que essas colunas devem ter o tipo **string**. Além disso, a coluna **instant\_bookable** deveria ter o tipo **bool**. Por fim, a coluna **last\_review** deveria ter um tipo de dados que melhor representa uma data.

```

[7]: string_cols = [
    'name',

```

```

    'host_name',
    'neighbourhood_group',
    'neighbourhood',
    'cancellation_policy',
    'room_type',
    'house_rules',
]

for col in string_cols:
    df[col] = df[col].astype('string')

df['instant_bookable'] = df['instant_bookable'].astype(bool)

df['last_review'] = pd.to_datetime(df['last_review'])

```

#### 1.2.4 Correção nos erros de digitação no nome dos bairros que apresentavam “brookln” e “manhatan”

```

[8]: df['neighbourhood_group'] = df['neighbourhood_group'].replace({
    'brookln': 'Brooklyn',
    'manhatan': 'Manhattan'
})

print(df['neighbourhood_group'].unique())

<StringArray>
['Brooklyn', 'Manhattan', <NA>, 'Queens', 'Staten Island', 'Bronx']
Length: 6, dtype: string

```

#### 1.2.5 Tratamento de Valores Ausentes.

##### Remoção de Anúncios sem preço

```

[9]: df.dropna(subset=['price', 'service_fee'], inplace=True)

print(f"Valores nulos em 'price' após remoção: {df['price'].isnull().sum()}")

```

Valores nulos em 'price' após remoção: 0

**Criação de coluna booleana para house\_rules** Como metade dos valores é nulo, iremos criar uma nova coluna para indicar se o anúncio possui ou não regras definidas.

```

[10]: df['has_house_rules'] = df['house_rules'].notna()

# Podemos agora remover a coluna original se o conteúdo de texto não for usado
# df_silver.drop(columns=['house_rules'], inplace=True)

```

```
print(df['has_house_rules'].value_counts())
```

```
has_house_rules
False      51174
True       49484
Name: count, dtype: int64
```

Preenchimento dos anúncios sem nome (ou sem nome de host) com “Sem nome informado”

```
[11]: for col in ['name', 'host_name']:
      df[col] = df[col].fillna('Sem nome informado')
```

Remoção dos demais nans

```
[12]: nans_to_drop = [
      'neighbourhood',
      'neighbourhood_group',
      'lat',
      'long',
      'host_identity_verified',
      'minimum_nights',
      'review_rate_number',
      ]

df.dropna(subset=nans_to_drop, inplace=True)
```

### 1.2.6 Tratamentos de valores inconsistentes

Foram identificados alguns valores negativos na coluna **availability\_365**. No entanto, não faz sentido que essa coluna tenha valores negativos, visto que ela indica uma quantidade de dias futuros. Ou seja: o valor mínimo deveria ser 0.

```
[13]: df = df[df['availability_365'] >= 0]
      print(df['availability_365'].min())
```

0

### 1.2.7 Tratamentos de Dados Duplicados

Na análise dos dados na camada bronze, percebemos que existem anúncios duplicados. Os anúncios duplicados são dois registros diferentes que possuem o mesmo id. Por isso, vamos remover as duplicatas.

```
[14]: print(f"Número de linhas TOTAIS no DataFrame inicial: {len(df)}")
      print("-" * 45)

      duplicated_lines = df[df.duplicated(subset=['id'], keep=False)]

      num_of_duplicated_ids = duplicated_lines['id'].nunique()
      print(f"Encontrados {num_of_duplicated_ids} IDs que se repetem.")
```

```

print(f"Esses IDs correspondem a um total de {len(duplicated_lines)} linhas no_
↳DataFrame.")

df_without_duplicates = df.drop_duplicates(subset=['id'], keep='first')

print(f"\nNúmero de linhas APÓS a remoção: {len(df_without_duplicates)}")
print(f"Cálculo da operação: {len(df)} (linhas iniciais) - {df.
↳duplicated(subset=['id']).sum()} (ocorrências extras) =_
↳{len(df_without_duplicates)}")
print("-" * 45)

number_of_duplicates_after_cleanup = df_without_duplicates.
↳duplicated(subset=['id']).sum()

print(f"Número de IDs duplicados no DataFrame final:_
↳{number_of_duplicates_after_cleanup}")

df = df_without_duplicates

```

Número de linhas TOTAIS no DataFrame inicial: 99986

-----

Encontrados 524 IDs que se repetem.

Esses IDs correspondem a um total de 1048 linhas no DataFrame.

Número de linhas APÓS a remoção: 99462

Cálculo da operação: 99986 (linhas iniciais) - 524 (ocorrências extras) = 99462

-----

Número de IDs duplicados no DataFrame final: 0

## 1.3 LOAD

### 1.3.1 Salvando um csv com os dados transformados

```

[15]: df.to_csv(data_layer_filepath + 'silver/airbnb-dataset-silver.csv', index=False)

print("Dataset da camada Silver salvo com sucesso!")

```

Dataset da camada Silver salvo com sucesso!

### 1.3.2 Carregando os dados na base de dados

```

[16]: import pandas as pd
from psychopg import connect, sql

print("--- Iniciando processo de carga e verificação no PostgreSQL ---")

DB_USER = "postgres"
DB_PASSWORD = "postgres"

```



```

DB_HOST = "localhost"
DB_PORT = "5433"
DB_NAME = "airbnb"
DB_SCHEMA = "silver"
TABLE_NAME = "listings"
TABLE_FULL_NAME = f"{DB_SCHEMA}.{TABLE_NAME}"

DDL = f"""
CREATE SCHEMA IF NOT EXISTS {DB_SCHEMA};
DROP TABLE IF EXISTS {TABLE_FULL_NAME};
CREATE TABLE {TABLE_FULL_NAME} (
    id BIGINT PRIMARY KEY, name TEXT, host_id BIGINT, host_identity_verified_
    ↪BOOLEAN,
    host_name VARCHAR(255), neighbourhood_group VARCHAR(255), neighbourhood_
    ↪VARCHAR(255),
    lat NUMERIC(10, 7), long NUMERIC(10, 7), instant_bookable BOOLEAN,
    cancellation_policy VARCHAR(100), room_type VARCHAR(100), construction_year_
    ↪INTEGER,
    price NUMERIC(10, 2), service_fee NUMERIC(10, 2), minimum_nights INTEGER,
    number_of_reviews INTEGER, last_review DATE, reviews_per_month NUMERIC(5,
    ↪2),
    review_rate_number NUMERIC(10, 2), calculated_host_listings_count INTEGER,
    availability_365 INTEGER, house_rules TEXT, has_house_rules BOOLEAN
);
"""

print(f"Total de linhas a serem carregadas: {len(df)}")

cols = list(df.columns)

insert_query = sql.SQL("INSERT INTO {} ({} ) VALUES ({} )").format(
    sql.Identifier(DB_SCHEMA, TABLE_NAME),
    sql.SQL(", ").join(map(sql.Identifier, cols)),
    sql.SQL(", ").join(sql.Placeholder() * len(cols))
)

with connect(f"host={DB_HOST} dbname={DB_NAME} user={DB_USER}_
    ↪password={DB_PASSWORD} port={DB_PORT}") as conn:
    print("\nConexão com o PostgreSQL estabelecida.")
    with conn.cursor() as cur:
        cur.execute(DDL)
        print("Estrutura do banco de dados criada com sucesso.")
        conn.commit()

    print("Iniciando carga de dados...")

    for _, row in df.iterrows():

```

```

        values = [None if pd.isna(v) else v for v in row]
        cur.execute(insert_query, values)

    conn.commit()
    print("Carga concluída com sucesso!")

    check_number_of_lines = f'select count(*) from {TABLE_FULL_NAME};'
    cur.execute(check_number_of_lines)
    for query_return in cur.fetchall():
        print(f'Foram carregadas {query_return[0]} linhas no banco de dados!
↵')

```

--- Iniciando processo de carga e verificação no PostgreSQL ---  
 Total de linhas a serem carregadas: 99462

Conexão com o PostgreSQL estabelecida.  
 Estrutura do banco de dados criada com sucesso.  
 Iniciando carga de dados...  
 Carga concluída com sucesso!  
 Foram carregadas 99462 linhas no banco de dados!

[ ]: